

SDP



Agentic SDLC

Designing the SDLC Your Agents Deserve

Five maturity levels — platform readiness, developer adoption, and governance that compound

"How do you know if your SDLC is ready for agents — and what do you build first?"

Most teams add agents to an SDLC designed for humans and wonder why throughput doesn't scale. The answer is not the agent — it is the pipeline around it.

Platform without adoption stalls at L3. Adoption without platform never reaches L5. Both must...

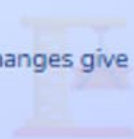
Developers

What does an agent-ready SDLC look like — and how do I know if mine qualifies?



Platform Engineers

Which CI, repo, and governance changes give agents the most productivity?



Engineering Leaders

How do I assess where we are today and build a credible roadmap to L5?



50× code volume

Habits for 300 lines/day collapse at 15,000 — the L5 breakpoint every team hits

96× faster

Same agent work: 3 days in a polyrepo vs. 45 min in a monorepo

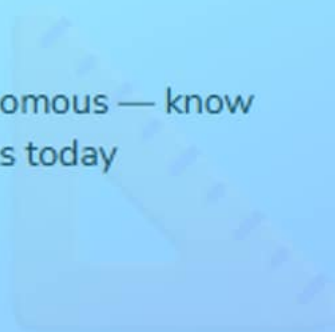
5× throughput

8-min CI delivers 15 features/day vs. 3/day at 60 min — same agents

PART 1

Maturity Model

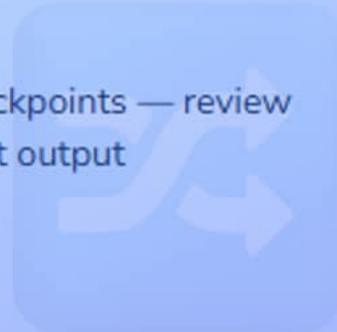
L1 Functional → L5 Autonomous — know exactly where your org sits today



PART 2

PR Workflows

22 manual gates → 4 checkpoints — review capacity scales with agent output



PART 3

Trust Manufacturing

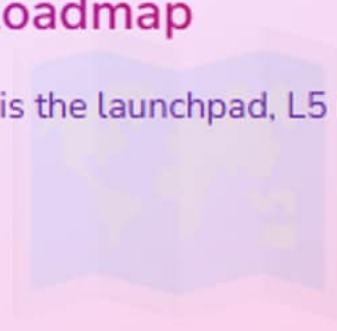
CI as trust factory — 10-min threshold, zero-flake, attestations



PART 4

Implementation Roadmap

Phase-by-phase path: L3 is the launchpad, L5 is the destination



Click any section to jump directly there

Maturity Model

Rate where your org sits today — then choose investments that close the gap.



Five Levels

L1 Functional → L5 Autonomous —
clear signals per level



Platform Signals

Tooling, MCP, CI speed, governance
gates — what you build



Adoption Signals

Usage, habits, instruction quality,
delegation — how teams use it

Assess → Govern → Trust → Scale

↳ Each section solves the bottleneck the previous one exposed

Five Levels of Agentic Readiness

L1 Functional	Build scripts run reliably, basic CI exists, no flaky failures	Baseline
L2 Documented	copilot-instructions.md versioned, Chat habitual, conventions explicit	Onboarding
L3 Standard	MCP ≥ 1 , CCR default, GHAS enabled, CCA delegated, MAU $\geq 60\%$	Standardized
L4 Optimized	MCP ≥ 2 enterprise, coding agents in PR flow, seat utilization $\geq 80\%$	Optimized
L5 Autonomous	Agents own features end-to-end, Metrics API + DORA correlated	Autonomous

The Volume Flip: Where Review Habits Collapse

Scarcity shifts from writing code to trusting it at velocity

50x

more code to review — habits designed for 300 lines/day collapse at 15,000

🔥 L3 assumption

Humans produce 300 lines/day — line-by-line review is feasible

🤖 L5 reality

Agents produce 15,000 lines/day — detail review is impossible

🔄 What must change

PR governance and CI speed both need L5-native redesign

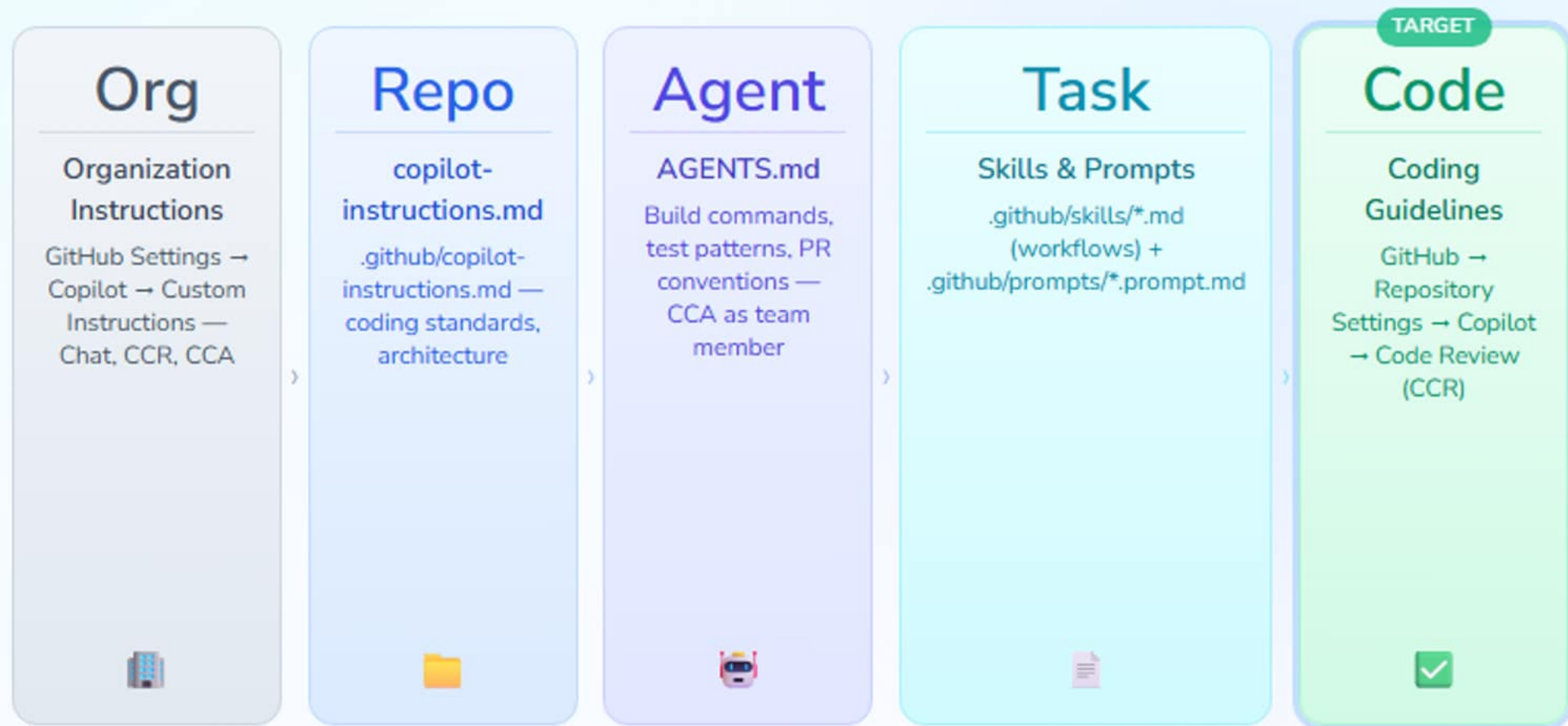
💡 The insight

The bottleneck is not writing code — it is manufacturing trust at velocity

🌀 At L5, scarcity moves from code production to trust production. Every section that follows addresses one layer of that shift.

Instruction Hierarchy: Highest-Leverage Cross-Cutting Investment

One investment that compounds at every maturity level — treat config as first-class code



Version in GitHub repos, review in PRs, evolve alongside the codebase — this is what separates good from excellent

PR Workflows

Coordination is solved — now review capacity is the bottleneck.



Governance Pyramid

22 manual gates → 4 checkpoints +
90% automated



Agent Validation

Context-aware: 44 real warnings vs
847 false positives



Human Judgment

Architects review architecture and
intent — not every line

Review capacity per engineer after governance automation
↳ 300 lines/day → 15,000 lines/day

How Review Mode Shifts at Each Level

L1 Functional	Manual verification — basic tooling, no Copilot habits	Manual
L2 Documented	Human review with Copilot Chat context — completions habitual	Assisted
L3 Standard	Repeatable gates: CCR on all PRs, branch rulesets, CCA delegated	Gated
L4 Optimized	Context-aware automation + human exception handling — MCP ≥ 2	Automated
L5 Autonomous	Outcome validation: 4 checkpoints, 90% automated — Metrics + DORA	Outcome

L3 Review Habits vs. L5 Agent Volume

Same governance model, 50x the volume — the math doesn't work

✗ L3 PR Review

- 1 Read every line
50–200 line change, 30-min review
- 2 Discussion threads
Why did you use a Map here?
- 3 Manual compliance
Security team reviews every auth change
- 4 LGTM after 3 hrs
300 lines/day/reviewer capacity

🔄 4–7 days PR to merge at L5 volume

✓ L5 PR Review

- 1 Validate intent
500–2000 lines — read requirements, not implementation
- 2 Evidence bundle check
Tests passed, security clean, coverage met
- 3 Automated gates
90% of checks automated — humans review exceptions
- 4 Approved after 20 min
15,000 lines/day/reviewer capacity

✓ 2–4 hours PR to merge

50x review capacity increase — same engineers, governance automation does the heavy lifting

Governance Pyramid Inversion: 22 Gates → 4 Human Checkpoints

Traditional Governance

22 manual approval gates

Security, compliance, architect, QA, lead — each per PR

Senior architect reviews every architecture decision

Security team manually reviews every auth change

Result

4–7 days from PR to merge

L5 Governance

4 human checkpoints

Intent, security risk, architectural fit, outcome validation

90% of checks automated — security, compliance, quality

Architects review violations and exceptions only

Result

2–4 hours from PR to merge

90%

automated enforcement

50×

review capacity per engineer

4 hrs

avg time to merge

💡 CODEOWNERS + required reviewers + automated status checks — humans judge architecture, intent, and strategic risk.

Context-Aware Validation: Judgment, Not Pattern Matching



Rules-Based Scan

847 warnings generated

Regex pattern fires on every match

803 false positives — test fixtures, docs, examples

Developers ignore all warnings

Signal lost in the noise

Actual PII violation ships to production



Agent Validation

44 real warnings generated

Agent reads context: fixture vs. production handler

2 false positives — 97% signal quality

Developers trust the signal

Every flag comes with reasoning

Actual violations caught before merge — always

Trust Manufacturing

Review scales — now you need CI to manufacture trust at velocity.



10-Minute Threshold

Agents idle 80% at 60-min CI,
productive 95% at 8-min CI



Zero-Flake Tolerance

Quarantine on first flake, fix in 2 days
— green means trustworthy



Attestations

SLSA evidence trails for regulated
environments and audit

Features per day: 60-min CI vs 8-min CI, same agents running continuously
↳ 3 features/day → 15 features/day

Manufacturing Principles Applied to CI

Repeatable	Same checks, same order, every time — reliable trust signals	Deterministic
------------	--	---------------

Automated	327 tests in 8 min, 10,000 security patterns in 45 sec	At Scale
-----------	--	----------

Evidence	Every check produces auditable, tamper-evident attestations	SLSA
----------	---	------

Fast	Agents idle 80% at 60-min CI, productive 95% at 8-min	< 10 min
------	---	----------

Zero Flake	Quarantine on first flake, fix in 2 days — green = trustworthy	100% Signal
------------	--	-------------

The 10-Minute Threshold: CI Speed Is Agent Productivity

Agents sit idle waiting for CI — faster feedback means higher throughput

5x

more features per day: 3/day at 60-min CI vs.
15/day at 8-min CI — same agents

🏃 60-minute CI

Agents idle 80% — 4 iteration cycles = 4 hours of wall time

⚡ 8-minute CI

Agents productive 95% — 4 iteration cycles = 52 minutes

🔧 How to get there

Affected analysis + parallelization + caching: test only what changed

📈 Cache economics

15th feature costs 2-min CI (cache hit) — marginal trust cost near zero

🎯 Target: <10 min for PR checks. Every minute over 10 is agent idle time, compounded across 15 PRs/day.

Implementation Roadmap

Trust is automated — now a phased plan from where you are to L5.



Phase 1: Standardized

L3 — where platform, adoption, and quality first compound



Phase 2: Optimized

L4 — enterprise governance, advanced agents, deliberate adoption



Phase 3: Autonomous

L5 — agents own features end-to-end; ROI measured and reported

Platform and adoption must advance together

↳ L3 is the launchpad — skip it and L5 collapses under its own weight

Phase 1: Standardized Foundation (L3)

🏗️ Infrastructure

MCP ≥ 1 enterprise system

Internal docs, DB schema, or CI feedback connected

CCR as default reviewer

Automatic on all PRs in target repositories

GHAS enabled

CodeQL, secret scanning, Dependabot on every PR

Repo topology

Consolidate to monorepo if >30% of features cross repo boundaries

👤 Adoption & Governance

copilot-instructions.md versioned

Reviewed in PRs like any other code change

Copilot MAU ≥ 60%

Chat + completions + CLI habitual, not experimental

CCA delegated ≥ 3–5 issues/sprint

Issue templates structured for agent success

≥ 5 custom coding guidelines co-authored with tech lead

📖 Instruction Hierarchy: establish org instructions + copilot-instructions.md + AGENTS.md — the foundation everything builds on.

Phase 2: Optimized Governance (L4)

Infrastructure

MCP ≥ 2 enterprise systems

Multi-step orchestration: CCA + MCP + custom instructions

Coding agents in PR flow

CCA PR merge rate $\geq 60\%$, time to merge trending down

Org-level governance

Enterprise Copilot settings deliberate, not defaults

Policy-gated PR automation deployed

Adoption & Governance

CCR tuned, false positives $< 20\%$

Custom coding guidelines ≥ 10 , iterated from feedback

Seat utilization $\geq 80\%$

Inactive seats reallocated; PRU governance in place

Copilot Memory enabled

Monthly curation cadence established

PR review turnaround reduced $\geq 30\%$ vs. pre-CCR baseline

Instruction Hierarchy matures: AGENTS.md + prompt files + skills covering top workflows, guidelines tuned from CCR patterns.

Phase 3: Autonomous Scale (L5)

Infrastructure

End-to-end feature delivery

Intent spec → agent implementation → CI trust → merge → deploy

CI trust factory operational

<10-min PR checks; zero-flake enforced; SLSA attestations

Governance pyramid inverted

4 checkpoints, 90% automated — 15,000 lines/day/reviewer

Policy-gated auto-merge live in production

Adoption & Measurement

Metrics API + DORA correlated

Copilot usage tied to deployment frequency and PR cycle time

ROI narrative produced

Leadership has quantified evidence: X% cycle time reduction

Champions program active

≥2 champions per team coaching peers and iterating instructions

Self-sufficiency score >4/5 — team sustaining and

 L5 is not a destination — it is the compounding result of L1–L4 doing their job. Miss any layer and the system bottlenecks.

What Changes When the SDLC Is Agent-Ready

Before

Line-by-line PR review — 300 lines/day per engineer

Flaky CI makes green builds untrustworthy signals

Coordination overhead across repos delays features

22 manual approval gates slow governance to days

After

Outcome-based review — 15,000 lines/day per engineer

Zero-flake tolerance — green always means trustworthy

Atomic monorepo changes eliminate coordination tax

4 human checkpoints replace 22 gates — hours not days

50×

review capacity per engineer

96×

faster to production (45 min vs 3 days)

5×

features per day at L5 vs L3 CI speed

✓ What You Can Do Today

Today

- Run the L1–L5 maturity assessment for your team — platform and adoption both

This Week

- Add or update `.github/copilot-instructions.md` and version it in a PR
- Enable Copilot Code Review as automatic reviewer on your primary repo
- Measure your CI p95 time — if it exceeds 10 min, you have a target

This Month

- Version `AGENTS.md` in every CCA-enabled repository
- Create prompt files for your top 3 repeatable workflows
- Evaluate monorepo migration if >30% of features cross repo boundaries
- Set a zero-flake policy — quarantine on first flake, fix in 2 days

🔑 Key Takeaway

L3 is the launchpad — every investment made there compounds at L4 and L5.

AgentRC Framework

[Microsoft AgentRC Repository](#)

Measure, generate, and maintain loop for repository AI readiness

[AgentRC Concepts and Maturity Model](#)

Maturity levels, readiness pillars, and how autonomy is staged

Security and Compliance

[SLSA Supply Chain Security Framework](#)

Attestation standards for regulated environments and audit trails

Infrastructure Tools

[Nx Monorepo Tools](#)

Build orchestration, module boundaries, and affected analysis

[GitHub Actions Documentation](#)

CI/CD workflow automation for agent-native pipelines

[GitHub Actions Caching Guide](#)

Dependency caching for faster CI feedback loops



Agentic SDLC

Designing the SDLC Your Agents Deserve

50x

more review capacity — governance
pyramid inverted, 22 gates → 4
checkpoints

96x

faster to production — monorepo
eliminates the coordination tax

5x

more features per day — 8-min CI keeps
agents productive 95% of the time

Which maturity level are you at today — and what is the one infrastructure investment that would move you to the next?